

El paradigma de orientación a objetos algoritmos

M4: Fundamentos de programación en Java

|AE5: Utilizar elementos de la programación orientada a objetos para la implementación de una pieza de software que da solución a un problema de baja complejidad

Introducción

En esta lección veremos la importancia de la programación orientada a objetos y cómo nos ayuda a crear programas más eficientes y estructurados.

Aprenderás las características, las propiedades y el comportamiento de los objetos. También estudiaremos los métodos de una clase, que son las acciones que un objeto puede realizar.

A medida que avancemos, comprenderemos cómo los objetos pueden colaborar entre sí y cómo podemos crear diferentes relaciones entre ellos.

Aprendizaje esperado

Al finalizar la lección podrás:

- Utilizar elementos de la programación orientada a objetos para la implementación de una pieza de software que da solución a un problema de baja complejidad.

La importancia de la Orientación a Objetos en la programación

La programación orientada a objetos (**POO**) es importante porque ofrece una forma estructurada y eficiente de desarrollar software. Permite organizar el código en módulos o **clases** reutilizables, lo que facilita la creación y mantenimiento de programas complejos.

Una de las principales ventajas de la POO es la capacidad de modelar el mundo real de manera más precisa. Los **objetos** en la POO representan entidades o conceptos del mundo real, y las interacciones entre ellos se reflejan a través de métodos y atributos. Esto facilita la comprensión y el diseño de programas, ya que se pueden utilizar conceptos familiares para abordar problemas de programación.

Clases y objetos

- **Clases:** son las plantillas o moldes que utilizaremos para crear objetos. Definen las propiedades y comportamientos que los objetos de esa clase tendrán.
- **Objetos:** son instancias específicas de una clase. Se crean a partir de la clase y tienen su propio estado y comportamiento.

Entonces, una clase es una especie de plantilla o prototipo de objetos: define los atributos que componen ese tipo de objetos y los métodos que pueden emplearse para trabajar con esos objetos.

En su forma más simple, una clase se define por la palabra reservada `class` seguida del nombre de la clase. El nombre de la clase debe empezar por mayúscula. Si el nombre es compuesto, cada palabra debe empezar por mayúscula.

La definición de la clase se pone entre las llaves de apertura y cierre.

```
public class NombreClase {
```

```
// atributos ;
```

```
// constructores ;
```

```
// métodos propios ;
```

}

Atributos de una clase y estado de un objeto

En términos generales, un objeto es una abstracción conceptual del mundo real que se puede traducir a un lenguaje de programación orientado a objetos. Los objetos del mundo real comparten dos características: **todos poseen estado y comportamiento**. Por ejemplo, el perro tiene estado (color, nombre, raza, edad) y comportamiento (ladrar, caminar, comer, sentarse). Por lo tanto, un **estado** permite informar cuáles son las características del objeto y lo que este representa, y el **comportamiento**, consiste en decir lo que sabe/puede hacer.

El estado de un objeto es una lista de variables conocidas como sus **atributos**, cuyos valores representan el estado que caracteriza al objeto.

El comportamiento es una lista de **métodos**, procedimientos, funciones u operaciones que un objeto puede ejecutar a solicitud de otros objetos. Los objetos también se conocen como **instancias**.

Atributos de una Clase

Los atributos **son características comunes a todos los objetos**. Son los "espacios" donde alojaremos información que cambiará en cada objeto pero que corresponda a una descripción del mismo.

El estado o información de un objeto se almacena en atributos. Los atributos pueden ser de tipos primitivos de Java o del tipo de otros objetos.

La declaración de un atributo de un objeto tiene la siguiente forma:

<tipo de dato> <nombre> ;

<tipo>: indica la clase a la que pertenece el atributo definido.

<nombre>: puede ser cualquier identificador válido y denomina el atributo que está siendo declarado.

Por ejemplo, si tenemos una **clase** "Coche" con **atributos** como "color", "marca" y "modelo", el **estado** de un objeto de esa clase podría ser "rojo", "Toyota" y "Corolla". Estos valores representan el estado actual del coche.

Métodos de una Clase

Los métodos de una clase son funciones o procedimientos asociados a dicha clase. **Representan el comportamiento o las acciones que los objetos de esa clase pueden realizar.** Los métodos encapsulan un conjunto de instrucciones que definen una tarea específica que se puede realizar sobre un objeto.

Los métodos pueden aceptar parámetros como entrada, realizar cálculos, modificar el estado de los objetos y devolver o no resultados como salida. Ayudan a organizar y modularizar el código, ya que permiten dividir la lógica del programa en tareas más pequeñas y manejables.

```
[<modificador>] <nombre de método> ( <argumento>* ) {  
    <sentencia>;  
}
```

Métodos constructores

Los constructores son métodos **propios del objeto** que permiten la creación de dicho objeto. A la creación de un objeto se le denomina **instanciación**.

Además de definir los atributos de un objeto, es necesario definir los métodos que determinan su comportamiento.

Los constructores se encargan de dar un **estado inicial** a nuestro objeto. Estos métodos tienen como característica principal **tener el mismo nombre que la propia clase**. Por lo tanto, debemos tener un método dentro de la clase “Coche” que se llame igual que ella.

```
[<modificador>] <nombre de clase> ( <argumento>* ) {  
    <sentencia>*  
}
```

```
public class Coche {
    public int id;
    public String marca;
    public String color;

    // constructor por defecto (sin parámetros)
    public Coche() {

    }

    //constructor con parámetros
    public Coche(int id, String marca, String color) {

        this.id = id;
        this.marca = marca;
        this.color = color;
    }
}
```

El método constructor se ejecuta cada vez que se instancia un objeto de la clase. Este método se utiliza para **inicializar** los atributos del objeto que se crea.

Para diferenciar entre los atributos del objeto y los identificadores de los parámetros del método constructor, se utiliza la palabra **this**. De esta forma, los parámetros del método pueden tener el mismo nombre que los atributos de la clase.

La **instancia** de un objeto consiste en asignar un espacio de memoria al que se hace referencia con el nombre del objeto. Los identificadores de los objetos permiten acceder a los valores almacenados en cada objeto.

El constructor por defecto

Cada clase tiene al menos un constructor. Si no se escribe un constructor, el lenguaje de programación **Java le provee uno por defecto**. Este constructor no posee argumentos y tiene un cuerpo vacío. Si se define un constructor que no sea vacío, el constructor por defecto se pierde, salvo que creamos un nuevo constructor vacío.

Una vez que se ha declarado una clase, se pueden crear objetos a partir de ella. A la creación de un objeto se le denomina **instanciación**. Por esta razón se dice que **un objeto es una instancia de una clase** y el término instancia y objeto se utilizan indistintamente. Para crear objetos, basta con declarar una variable de alguno de los tipos de las clases definidas.

```
NombreClase nombreObjeto;
```

Para crear el objeto y asignar un espacio de memoria es necesario utilizar el **operador new**. El operador new instancia el objeto y reserva espacio en memoria para los atributos y devuelve una referencia que se guarda en la variable.

```
nombreObjeto = new nombreClase();
```

Tanto la declaración de un objeto como la asignación del espacio de memoria se pueden realizar en un solo paso:

```
Coche miAuto = new Coche();
```

A partir de este momento los objetos ya pueden ser referenciados por su nombre (miAuto).

Abstracción

La abstracción es la propiedad que considera los aspectos más significativos o notables de un problema y expresa una solución en esos términos. La abstracción posee diversos grados o niveles de abstracción, los cuales ayudan a estructurar la complejidad intrínseca que poseen los sistemas del mundo real. La abstracción encarada desde el punto de vista de la programación orientada a objetos **es el mecanismo por el cual se proveen los límites conceptuales de los objetos y se expresan sus características esenciales**, dejando de lado sus características no esenciales. Si un objeto tiene más características de las necesarias los mismos resultan difíciles de usar, modificar, construir y comprender. En el análisis hay que concentrarse en ¿Qué hace? y no en ¿Cómo lo hace?.

Encapsulamiento

El encapsulamiento es el proceso de agrupamiento de datos y operaciones relacionadas bajo una misma unidad de programación, lo que permite aumentar la cohesión de los componentes del sistema.

El encapsulamiento oculta lo que hace un objeto de lo que hacen otros objetos y del mundo exterior, por lo que se denomina también ocultación de datos.

Un objeto tiene que presentar “una cara” al mundo exterior de modo que se puedan iniciar sus operaciones. Los métodos operan sobre el estado interno de un objeto y sirven como el mecanismo primario de comunicación entre objetos. **Ocultar el estado interno y hacer que toda interacción sea a través de los métodos del objeto es un mecanismo conocido como encapsulación de datos.**

Modificadores de acceso

Para lograr el uso correcto del encapsulamiento vamos a utilizar los modificadores de acceso, que van a permitirnos elegir cómo se accede a los datos y a través de qué.

Todas las clases poseen diferentes niveles de acceso en función del modificador de acceso (**visibilidad**). A continuación, se detallan los niveles de acceso con sus símbolos correspondientes:

Public: Este modificador permite acceder a los elementos desde cualquier clase, independientemente de que ésta pertenezca o no al paquete en que se encuentra el elemento.

Private: Es el modificador más restrictivo y especifica que los elementos que lo utilizan sólo pueden ser accedidos desde la clase en la que se encuentran. Este modificador sólo puede utilizarse sobre los atributos de una clase y sobre interfaces y clases internas, no sobre clases o interfaces de primer nivel, dado que esto no tendría sentido. Es importante destacar también que el modificador `private` convierte los elementos en privados para otras clases, no para otras instancias de la clase. Es decir, un objeto de una determinada clase puede acceder a los atributos privados de otro objeto de la misma clase.

Protected: Este modificador indica que los elementos sólo pueden ser accedidos desde su mismo paquete y desde cualquier clase que extienda (herede) la clase en que se encuentra, independientemente de si esta se encuentra en el mismo paquete o no. Este modificador, como `private`, no tiene sentido a nivel de clases o interfaces no internas.

Accesadores y Mutadores

Para acceder a los atributos de un objeto se definen los métodos **get** y **set**.

Los métodos **get** (**accesadores**) se utilizan para consultar el estado de un objeto, mientras que los métodos **set** (**mutadores**) para modificar su estado.

Un método `get` se declara **public** y a continuación se indica el tipo de dato que devuelve. Es un método de consulta. En el cuerpo del método se utiliza **return** para devolver el valor correspondiente al atributo al cual se hace referencia en el constructor como `this.nombreAtributo`.

```
public String getMarca() {
```



```

    return marca;
}

```

Por otra parte, un método **set** se declara **public** y **retorna void**. La lista de parámetros de un método set incluye el tipo y el valor a modificar. Es un método modificador. El cuerpo de un método set asigna al atributo del objeto el valor del parámetro de la declaración.

```

public void setMarca(String marca) {
    this.marca = marca;
}

```

Los accedadores y mutadores, junto con los modificadores de acceso, **permiten establecer un nivel adecuado de control y seguridad en el acceso y modificación de los atributos de una clase**. Ayudan a mantener la coherencia y consistencia de los datos, además de proporcionar una interfaz más clara y segura para interactuar con los objetos de una clase.

Colaboración entre objetos - Relaciones entre Clases

En la programación orientada a objetos, un objeto se comunica con otro para utilizar la funcionalidad y los servicios proporcionados por ese objeto.

Una **relación** es una conexión semántica entre clases. Permite que una clase conozca sobre los atributos, operaciones y relaciones de otras clases.

Las clases no actúan aisladas entre sí, al contrario, las clases están relacionadas unas con otras. Una clase puede ser un tipo de otra clase —generalización— o bien puede contener objetos de otra clase de varias formas posibles, dependiendo de la fortaleza de la relación entre las dos clases.

La relación entre dos clases separadas se establece a través de sus objetos. Es decir, las clases se conectan conceptualmente. Las relaciones entre clases significan que una clase contiene una referencia a un objeto (o varios) de la otra clase en la forma de un atributo.

Por ejemplo: un objeto **curso** tiene varios objetos **alumnos** y un objeto **profesor**. Esto significa que, gracias a la relación entre objetos, el objeto curso puede tener

toda la información que necesita. Además, nos ayuda para la reutilización de código.

Asociación

En las relaciones de asociación se puede establecer una relación bidireccional, donde los objetos que están al extremo de una relación pueden “conocerse” entre sí, o una relación unidireccional donde solamente uno de ellos “conoce” al otro.

Dentro de la asociación simple existe la composición y la agregación, que son las dos formas de relaciones entre clases.

Agregación

Representa un tipo de relación muy particular, en la que **una clase es instanciada por otro objeto y clase**. La agregación se podría definir como el momento en que dos objetos se unen para trabajar juntos y alcanzar una meta.

Un punto muy importante es que **ambos objetos son independientes entre sí**. En agregación, ambos objetos pueden sobrevivir individualmente, lo que significa que al borrar un objeto no afectará a la otra entidad.

Para validar la agregación, la frase “**Usa un**” debe tener sentido, por ejemplo: El programador usa una computadora. El objeto computadora va a existir más allá de que exista o no el objeto programador.

Composición

La composición es una relación más fuerte que la agregación, es una "relación de vida", es decir, que **el tiempo de vida de un objeto está condicionado por el tiempo de vida del objeto que lo incluye**. La composición es un tipo de relación dependiente en donde un objeto más complejo es conformado por objetos más pequeños.

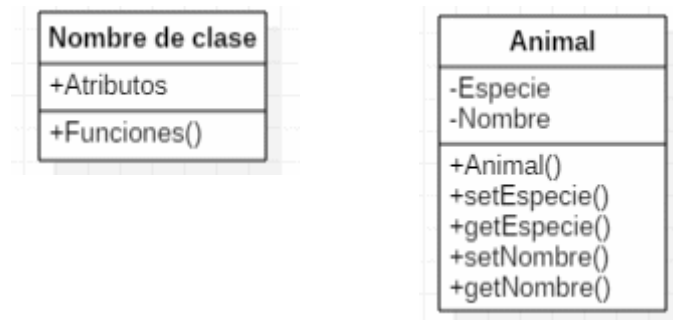
En esta situación, la frase “**Tiene un**”, debe tener sentido, por ejemplo: el cliente tiene una cuenta bancaria. Esta relación es una composición, debido a que al eliminar el cliente la cuenta bancaria no tiene sentido, y también se debe eliminar, es decir, la cuenta existe sólo mientras exista el cliente.

Diagrama de clases

Estos diagramas se utilizan para representar la estructura de un programa mostrando sus clases, métodos y atributos.

Los diagramas de clases también nos ayudan a identificar la relación entre diferentes clases u objetos. Ya sean relaciones entre clases o herencia.

Cada clase está representada por un rectángulo que tiene una subdivisión de tres compartimentos: nombre, atributos y métodos.

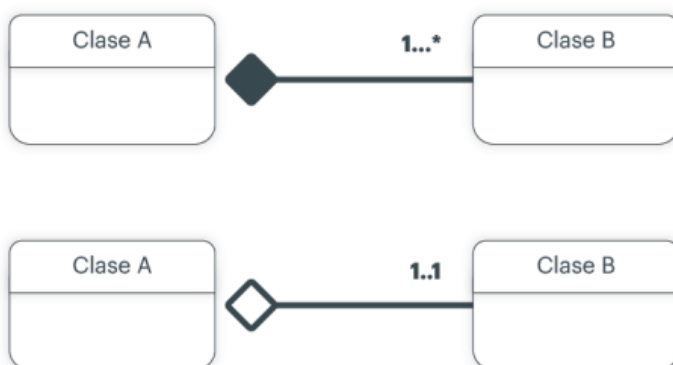


Hay tres tipos de modificadores que se utilizan para decidir la visibilidad de atributos y métodos:

- “+” se usa para el modificador de acceso public.
- “#” se usa para el modificador de acceso protected.
- “-“ se usa para el modificador de acceso private.

Las relaciones entre clases se representan con flechas entre las clases. La clase que recibe la relación de la otra clase (como un objeto) es la clase hacia donde señala el rombo.

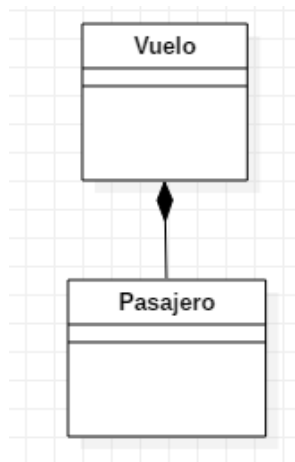
Para representar el tipo de relación, ya sea uno a uno o de uno a muchos, utilizamos un símbolo para cada relación en la flecha.



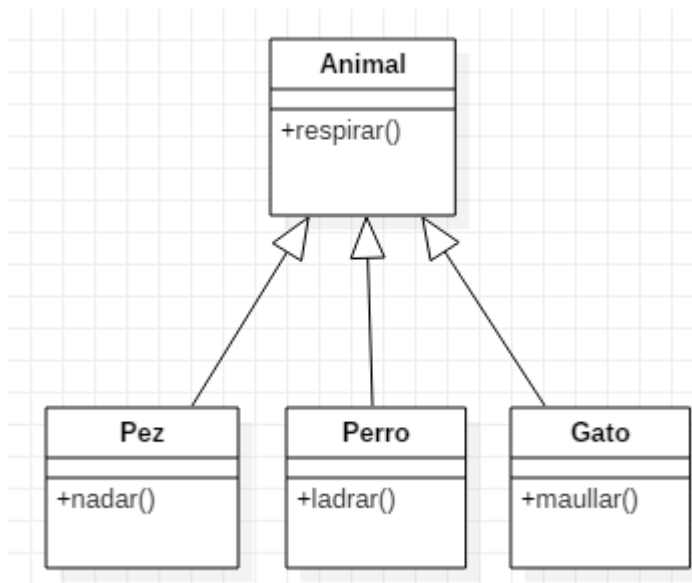
Ejemplo de agregación (rombo blanco):



Ejemplo de composición (rombo negro):



Ejemplo de herencia (flecha común):



Cierre

A lo largo de la lección, hemos comprendido la importancia de la Programación Orientada a Objetos en el desarrollo de software y cómo las clases y objetos son la base fundamental para representar entidades y sus características. Exploramos la definición de atributos y métodos para dar forma al estado y comportamiento de los objetos, junto con la relevancia de los métodos constructores para su inicialización adecuada. Además, discutimos sobre el acceso y modificación de datos mediante accesadores, mutadores y modificadores de acceso, asegurando una encapsulación apropiada.

En el mundo real, la programación orientada a objetos cobra vida mediante la colaboración entre objetos, permitiendo la interacción y comunicación entre distintas entidades del sistema. La composición, en particular, nos ha revelado cómo las clases pueden contener instancias de otras clases, fomentando una relación de "todo" y "parte".

La orientación a objetos nos proporciona un enfoque intuitivo y organizado para enfrentar los desafíos de la programación, permitiéndonos crear aplicaciones más eficientes y escalables.

¡Nos vemos de nuevo, más adelante! 

Referencias

- DesarrolloWeb.com. (2015, 12 de mayo). POO: Fundamentos de la Programación Orientada a Objetos. En Teoría de la Programación Orientada a Objetos. Recuperado de <https://desarrolloweb.com/articulos/poo-fundamentos-luis-fernandez.html>
- JavaAtPoint. (s.f.). useDelimiter (Scanner) en Java. Recuperado de <https://www.javatpoint.com/java-scanner-usedelimiter-method-in-java-with-examples>
- ProgramarYa. (s.f.). Clases y objetos (Java). Recuperado de <https://www.programarya.com/java/clases-y-objetos>
- Garro, A. (2014, 15 de abril). Capítulo 17: Otras relaciones entre objetos. Recuperado de <https://www.arkaitzgarro.com/java/capitulo-17.html>
- Plynko, P. (2023, 7 de junio). Accessors and Mutators in Java. CodeGym. Recuperado de <https://codegym.cc/groups/posts/accessors-and-mutators-in-java>

¡Muchas gracias!

Nos vemos en la próxima lección

